

INDIVIDUAL PROJECT REPORT

ON

“CampusSphere: Authentication, Access Control, Events and Venue Booking Engine for a Unified College Event and Activity-Points Platform”

Submitted in partial fulfillment of the requirement for the award of Degree of Bachelor of
Technology in Computer Science & Engineering

Submitted to:



Rajasthan Technical University, Kota (Raj.)

Submitted By:

Garvita Jain

(Roll No.: _____)

Member, Group Project CampusSphere

Under the guidance of

PROJECT GUIDE

Er. Ram Babu Buri

Department of Computer Science & Engineering



Session: 2026-27

Department of Computer Science & Engineering

**ARYA COLLEGE OF ENGINEERING & I.T.
SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR**

CERTIFICATE

This is to certify that the individual project report entitled “**CampusSphere: Authentication, Access Control, Events and Venue Booking Engine for a Unified College Event and Activity-Points Platform**”, submitted by **Garvita Jain (Roll No.: _____)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering of Rajasthan Technical University, Kota**, is a record of the bonafide work carried out by her/him as a member of the project group (Govind Jangid, Garvita Jain, Ayushi Garg, Ankit Yadav and Anshika Pathak) at **Arya College of Engineering & I.T., Jaipur** during the session 2026-27, under my guidance and supervision.

To the best of my knowledge, the work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Jaipur

Er. Ram Babu Buri
Project Guide
Dept. of CSE, ACEIT, Jaipur

Dr. Akhil Pandey
Head of Department
Dept. of CSE, ACEIT, Jaipur

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Er. Ram Babu Buri**, for his continuous guidance, encouragement and valuable suggestions throughout the project. His insistence on clear requirements, a weekly plan and honest review helped me understand how a large software system is planned and delivered by a team.

I sincerely thank **Dr. Akhil Pandey, Head of the Department of Computer Science and Engineering, Arya College of Engineering & I.T.**, for the academic support and the environment provided for this work. I am also grateful to all the faculty members of the **Department of Computer Science and Engineering** for their guidance and cooperation.

I would like to acknowledge the support of my teammates, **Govind Jangid, Ayushi Garg, Ankit Yadav and Anshika Pathak**, with whom I worked on **CampusSphere**. Our discussions on interfaces between the frontend, backend and database sections, the joint review of the specification and the shared planning of the 14-week schedule greatly enriched my understanding of team-based software development. I especially thank **Govind Jangid**, our Team Leader, for coordinating the weekly meetings and the integration of all five sections.

I also thank the faculty coordinators and student leads of the fifteen clubs of Arya College, whose experience with organising events shaped the requirements of the platform.

Finally, I extend my heartfelt gratitude to my family and friends for their constant support, patience and motivation throughout the course of this project.

Garvita Jain
(Roll No.: _____)
Computer Science & Engineering

ABSTRACT

This report presents my individual contribution to the group project **CampusSphere**, a unified web platform for college events, club treasuries, verified attendance and AICTE activity points at **Arya College of Engineering & I.T., Jaipur**. The work was divided among five students into two frontend and deployment sections (Govind Jangid and Ankit Yadav) and three backend and database sections (Garvita Jain, Ayushi Garg and Anshika Pathak).

My section is the backend foundation on which the other modules stand. It covers registration and login with BCrypt-hashed passwords, access tokens of 15 minutes and refresh tokens of 7 days stored hashed and rotated, role-based access control for the STUDENT, CLUB_ADMIN and SUPER_ADMIN roles with club scoping, the clubs and treasury read APIs, the event proposal and approval workflow, the venue buffer collision engine that enforces a 30-minute setup and teardown gap, the immutable audit log, rate limiting and other security hardening, the transcript PDF with Dean sign-off and the recommender scoring API.

The report explains the technologies and concepts used, the design of each module including the interval-overlap algorithm of the venue engine and the token rotation scheme, my 14-week plan, the dependencies on my teammates, the test plan with an authorisation matrix, and the challenges anticipated with their planned solutions.

This is a design and planning report. The targets quoted, such as response times, are goals to be measured during testing and the pilot, not measured results. The work provides practical experience in secure backend design, access control, workflow modelling and algorithm design.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION1

- 1.1 Project Overview1
- 1.2 Team and Work Division1
- 1.3 My Role and Responsibilities1
- 1.4 Objectives of My Section2
- 1.5 Project Duration and Schedule2
- 1.6 Tools and Collaboration3

CHAPTER 2 TECHNOLOGIES AND CONCEPTS4

- 2.1 Backend Framework and Layering4
- 2.2 JSON Web Tokens4
- 2.3 Refresh Tokens and Rotation4
- 2.4 BCrypt Password Hashing4
- 2.5 Role-Based Access Control4
- 2.6 Input Validation and DTOs5
- 2.7 Rate Limiting and Security Headers5
- 2.8 Audit Logging5
- 2.9 Interval Overlap and Indexing5
- 2.10 Workflow State Machines5
- 2.11 PDF Generation5
- 2.12 Similarity-Based Recommendation5
- 2.13 API Documentation and Testing6

CHAPTER 3 INDUSTRY APPLICATIONS AND RELEVANCE7

- 3.1 Identity and Access Management7
- 3.2 Secure Session Handling in Web Applications7
- 3.3 Resource Booking Systems7
- 3.4 Approval Workflows7
- 3.5 Audit and Compliance7
- 3.6 Recommendation Systems7
- 3.7 Future Scope7

CHAPTER 4 MY WORK: DESIGN AND IMPLEMENTATION APPROACH8

- 4.1 Module Structure8
- 4.2 Registration and Login APIs8
- 4.3 Token Issuance and Rotation8
- 4.4 RBAC Enforcement and Club Scoping8
- 4.5 Clubs API and Treasury Read Endpoint9
- 4.6 Event Proposal and Approval Workflow9
- 4.7 Venue Buffer Collision Engine10
- 4.8 Audit Logging11
- 4.9 Security Hardening11
- 4.10 Audit and Ledger Read APIs for the Super Admin12
- 4.11 Transcript PDF and Dean Sign-Off12
- 4.12 Recommender Scoring API12
- 4.13 Error Handling and Response Envelope12
- 4.14 Testing Approach12
- 4.15 Anticipated Challenges and Planned Solutions13
- 4.16 Integration with Teammates13

CHAPTER 5 14-WEEK WORK PLAN14

- 5.1 My Week-by-Week Plan14
- 5.2 Team Overview and Dependencies14
- 5.3 Reporting and Review15

CHAPTER 6 LEARNING OUTCOMES AND SKILLS16

6.1 Technical Skills16

6.2 Professional Skills16

6.3 Teamwork16

6.4 Problem Solving and Debugging16

6.5 Version Control and Communication16

CHAPTER 7 CONCLUSION18

REFERENCES19

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The group project **CampusSphere** is a unified platform for college events, club treasuries, verified attendance and AICTE activity points at Arya College of Engineering & I.T., Jaipur. It replaces chat-group announcements, form-based registration, personal UPI collection, paper attendance and static certificates with one authenticated system for Students, Club Admins and the Super Admin (Dean/HOD). The platform includes a directory of fifteen clubs with dedicated treasuries, an event approval workflow, venue booking with a 30-minute buffer, atomic seat reservation, Razorpay payments credited to club ledgers, a rolling 30-second QR pass, a camera-based gate scanner, attendance-gated feedback, SHA-256-sealed certificates and an automated activity-points ledger.

The project was carried out by five students over 14 weeks under the guidance of **Er. Ram Babu Buri**. This individual report describes the part assigned to me: the backend for identity and security (JWT with refresh rotation, BCrypt and role-based access control), the clubs and treasury read APIs, the event proposal and approval workflow, the venue buffer collision engine, audit logging, security hardening, the transcript PDF and the recommender scoring API.

1.2 Team and Work Division

The work was divided into five sections. Two members, Govind Jangid and Ankit Yadav, took the frontend and deployment. Three members, Garvita Jain, Ayushi Garg and Anshika Pathak, took the backend and the database. Table 1.1 shows the division.

Table 1.1: Division of work among the team

Member	Section	Responsibility in brief
Govind Jangid (Team Leader)	Frontend (student experience)	Student screens, rolling QR pass, checkout, certificates and points UI, AI feed, 3D twin; integration lead
Ankit Yadav	Frontend (admin consoles) and deployment	Club and Super Admin consoles, gate scanner PWA, Kanban, Docker, NGINX, CI/CD
Garvita Jain	Backend	Authentication, RBAC, clubs, events, venue buffer engine, audit, security
Ayushi Garg	Backend	Registration, capacity lock, payments, ledger, QR token service, certificate PDF
Anshika Pathak	Database and backend	Schemas, indexes, seeding, transactions, attendance, feedback, seal and points

1.3 My Role and Responsibilities

My role is the **backend developer for identity, security, clubs, events and venues**. I was assigned the following modules:

- registration, login, logout and token refresh, with BCrypt password hashing (cost 12) and JWT access and refresh tokens;
- role-based access control and club scoping for all endpoints;
- the clubs API and the read-only club treasury endpoint;
- the event proposal API and the approval workflow (DRAFT, PENDING_APPROVAL, APPROVED or REJECTED, then LIVE and COMPLETED) with a scheduler for status changes;
- the venue catalog and the 30-minute buffer collision engine with alternative venue suggestions;
- the immutable audit log with 365-day retention;
- security hardening: rate limits, validation, CORS and security headers;
- audit and ledger read APIs for the Super Admin finance console;
- the transcript PDF generator and the Dean sign-off API;
- the recommender scoring API for personalised events.

1.4 Objectives of My Section

- To authenticate users securely and issue tokens that are short lived, signed and revocable.
- To enforce authorisation on every service method, never relying on the user interface.
- To make sure a Club Admin can reach only the assigned club and a student can never obtain an administrator role.
- To make double booking of a venue impossible, including the setup and cleanup time.
- To record every approval and sensitive action in a log that cannot be edited.
- To provide stable, well-documented API contracts early so that the frontend can work in parallel.

1.5 Project Duration and Schedule

The project is planned over 14 weeks and follows the ten-phase roadmap: foundation and authentication (Weeks 1-2), clubs (Week 3), events and venues (Weeks 4-5), registration and rolling QR (Week 6), payments (Week 7), gate scanner (Week 8), feedback (Week 9), certificates and points (Week 10), AI suite (Week 11), 3D twin and deployment (Week 12), integration testing (Week 13) and pilot with documentation (Week 14). My modules form the foundation, so authentication, clubs, events and venues are scheduled in Weeks 1 to 6, and my later weeks go to security, audit, transcript and recommender work and to testing. My week-by-week plan is in Chapter 5.

1.6 Tools and Collaboration

I use the selected backend stack (Spring Boot 3.3 with Spring Security, or Node.js with Express or NestJS), MongoDB with the document mapping library, Postman and the OpenAPI (Swagger) document for contract testing, JUnit with MockMvc or Jest with Supertest for tests, Git and GitHub with pull requests, and the team issue board. The team communicates through a weekly meeting.

CHAPTER 2

TECHNOLOGIES AND CONCEPTS

This chapter explains the technologies and concepts that I use for the backend modules assigned to me.

2.1 Backend Framework and Layering

The backend uses a layered design: controllers receive HTTP requests and return the standard response envelope, services hold business rules and repositories access MongoDB. Spring Boot or NestJS provide dependency injection so that services can be tested with mock repositories. Controllers contain no business rules, which keeps the code easy to test and allows modules to be separated later.

2.2 JSON Web Tokens

A JSON Web Token is a signed compact token that carries claims. CampusSphere access tokens contain the user identifier (`sub`), role, roll number, administered clubs, issue time and expiry, signed with HS512 or RS256. Because the signature protects the claims, the server can authorise a request without a session store, which makes the application stateless and easy to scale.

2.3 Refresh Tokens and Rotation

Short access tokens limit the damage of theft, but users should not log in every 15 minutes. A refresh token with a 7-day life solves this. It is a random value stored only as a hash in the database and delivered through an `HttpOnly`, `Secure`, `SameSite=Strict` cookie. At each refresh the old token is invalidated and a new one issued (rotation). If an already-used refresh token appears again, it signals theft, and the whole token family is revoked.

2.4 BCrypt Password Hashing

BCrypt is a slow, salted hashing function designed for passwords. The work factor (cost) of 12 makes each guess expensive for an attacker but takes only a fraction of a second for a legitimate login. The cost can be raised as hardware improves.

2.5 Role-Based Access Control

RBAC grants permissions to roles. CampusSphere has three roles. Method-level annotations such as `hasAuthority('SUPER_ADMIN')` protect service methods. In addition to the role,

many operations need a scope check, for example that the club identifier of an event belongs to the `clubs` claim of a Club Admin.

2.6 Input Validation and DTOs

Requests are converted to data transfer objects with declared constraints (required fields, lengths, formats such as the roll number pattern, numeric ranges). Fields that must never be set by the client, such as `role`, `status` or `activity_points_total`, do not exist in the public DTOs, which removes the simplest forms of mass-assignment attack.

2.7 Rate Limiting and Security Headers

A token bucket allows a fixed number of requests per time. The limits are 100 requests per minute in general and 5 per minute for authentication endpoints. Security headers and a strict CORS list limit what browsers will accept from other origins. These follow common web security guidance (OWASP).

2.8 Audit Logging

An audit entry records who did what, when, from which IP address and what changed (a diff). Sensitive operations such as event approvals, rejections, manual overrides and role changes always write one. A TTL index removes entries after 365 days. Application code can only insert entries, never update or delete them.

2.9 Interval Overlap and Indexing

Two time intervals $[a,b)$ and $[c,d)$ overlap when $a < d$ and $c < b$. The venue engine extends the requested interval by the buffer and applies this test against existing events of the same venue. A compound index on venue and start time lets MongoDB examine only nearby bookings instead of the whole collection.

2.10 Workflow State Machines

An event moves through a fixed set of states and only certain transitions are allowed. Modelling this as a state machine in one service prevents invalid jumps, for example from DRAFT directly to LIVE, and makes the allowed transitions easy to test.

2.11 PDF Generation

The transcript is generated as a vector PDF with PDFKit or iText, with the college header, a summary table, an itemised log and a QR code that points to the public verification URL.

2.12 Similarity-Based Recommendation

A recommender compares a student's interest vector with each event's tag vector using cosine similarity and combines it with a department match and past attendance. The weights are configurable. The formula is simple and explainable, which is important for a student-facing feature.

2.13 API Documentation and Testing

The API is described with OpenAPI so the frontend can generate types and mocks. Unit tests cover services; MockMvc or Supertest tests call controllers with different tokens to prove that every endpoint behaves correctly for every role.

CHAPTER 3

INDUSTRY APPLICATIONS AND RELEVANCE

3.1 Identity and Access Management

Every modern system relies on authentication and authorisation. Industry practice, such as short-lived tokens, hashed passwords, refresh rotation and role-based permissions, is applied directly in this project.

3.2 Secure Session Handling in Web Applications

Banks, e-commerce companies and campus portals use token-based sessions and rate-limited login endpoints. The design shows how these techniques protect a small institutional system.

3.3 Resource Booking Systems

Hotels, meeting rooms, airlines and cloud providers must prevent double booking. Conflict detection with buffers is also used by operating theatres, studios and sports facilities.

3.4 Approval Workflows

Organisations use multi-step approvals with an audit trail for purchase orders, leave and publishing. The event approval flow is a small example of the same pattern.

3.5 Audit and Compliance

Regulated industries must keep tamper-evident logs of important actions. The audit log gives the Dean accountability over events, finance and overrides.

3.6 Recommendation Systems

Streaming, shopping and learning platforms personalise content. A simple explainable recommender is a good first step for an institutional platform.

3.7 Future Scope

The backend can be extended with single sign-on with the college e-mail, multi-factor authentication for administrators, fine-grained permissions per club, a deny-list for immediate role revocation, calendar integration and machine-learning-based ranking.

CHAPTER 4

MY WORK: DESIGN AND IMPLEMENTATION APPROACH

This chapter describes the design of the modules assigned to me. It is based on the project specification suite and on the schedule in Chapter 5.

4.1 Module Structure

My modules are organised as separate packages: `auth`, `rbac`, `clubs`, `events`, `venues`, `audit`, `transcript` and `recommender`. Each package has a controller, a service, a repository and its DTOs. Services call each other only through interfaces. The response envelope, error codes and the exception handler are shared so that every endpoint returns the same shape.

4.2 Registration and Login APIs

`POST /auth/register` accepts `name`, `institutional_email`, `password`, `roll_number`, `department`, `semester` and `interests`. The service validates the formats, checks that the email and roll number are unique, hashes the password with BCrypt (cost 12) and stores the user with the role `STUDENT`. The role is never read from the request. `POST /auth/login` verifies the password and returns the access token with `expires_in` 900 seconds, and sets the refresh cookie. After repeated failures the rate limit of five requests per minute applies. Error messages for wrong email and wrong password are the same so that attackers cannot find which emails exist.

4.3 Token Issuance and Rotation

Table 4.1: Token handling rules

Situation	Behaviour
Login success	Issue access token (15 min) and refresh token (7 days); store refresh hash with family identifier
Refresh with valid token	Invalidate used token; issue new pair in the same family
Refresh with already-used token	Treat as theft; revoke the entire family; require login
Logout	Revoke refresh token and clear cookie
Password change	Revoke all refresh tokens of the user
Expired access token	401; the client calls refresh and retries

4.4 RBAC Enforcement and Club Scoping

Every service method declares the required authority. For club-scoped operations the service compares the club identifier of the target resource with the `clubs` claim of the caller. For

financial operations, the claim is re-validated against the database so that a demoted administrator is stopped before the 15-minute token expires. The authority matrix is shown in Table 4.2. Any denied request returns 403 and writes an audit record.

Table 4.2: Authority matrix enforced in the services

Role	Allowed operations
STUDENT	Read public data; register; pay; read own tickets, points and certificates; submit feedback
CLUB_ADMIN	All student operations plus: manage own club profile; create and edit own-club events; check venues; operate the scanner; read own treasury; manage suggestions
SUPER_ADMIN	Everything above across all clubs plus: approve or reject events; read all ledgers and the audit log; override venue conflicts; sign off transcripts

4.5 Clubs API and Treasury Read Endpoint

`GET /clubs` is public and supports a category filter. `GET /clubs/:id/treasury` is restricted to the Club Admin of that club or the Super Admin. It returns the totals (total revenue, available balance, pending settlement, payout UPI identifier) and recent ledger entries. There is no write endpoint for balances: they are changed only by the payment settlement transaction implemented by Ayushi Garg and Anshika Pathak. Profile update endpoints accept only description, logo, banner, social links and leads, and verify the scope.

4.6 Event Proposal and Approval Workflow

`POST /events` accepts the proposal from a Club Admin: club, title, category, tags, venue, summary, description, banner, times, registration type, team size limits, price, capacity and proposed points. The service validates that the club belongs to the caller, checks the venue for conflicts, generates a unique slug and saves the event with status `PENDING_APPROVAL`. `PATCH /events/:id/status` is allowed for the Super Admin only. It accepts `APPROVED` or `REJECTED` with comments; the comment is mandatory for rejection. The allowed transitions are shown in Figure 4.1. A scheduler moves `APPROVED` events to `LIVE` at their start time and to `COMPLETED` at their end time, and the completion triggers the points engine of Anshika Pathak.

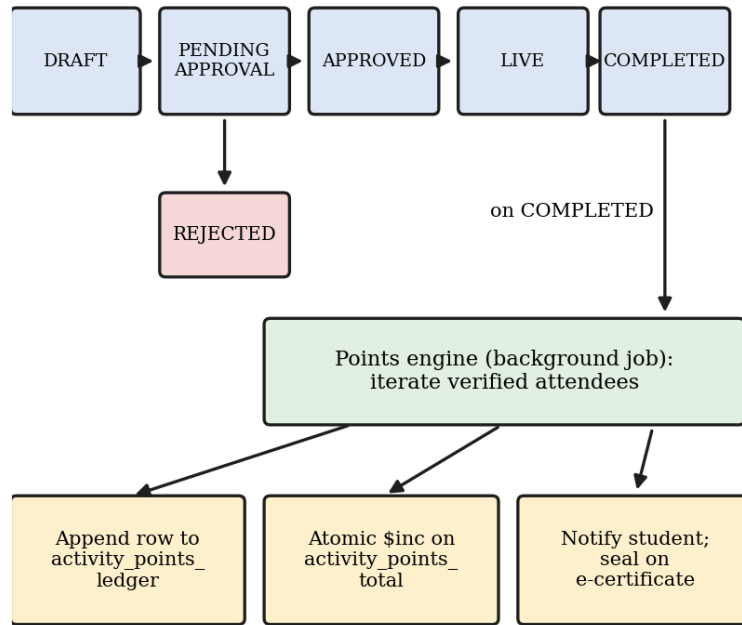


Figure 4.1: Event status transitions managed by the workflow service

4.7 Venue Buffer Collision Engine

Venues have a code, name, capacity, location and facilities. A booking request has a venue and a start and end time. The engine extends the requested interval by 30 minutes on both sides and looks for approved or live events of the same venue that overlap the extended interval:

$$\text{conflict} \iff \text{new.start} - 30 \text{ min} < \text{existing.end} \text{ and } \text{new.end} + 30 \text{ min} > \text{existing.start} \quad (1)$$

```

{ venue_id: V,
  status: { $in: ['APPROVED','LIVE'] },
  'event_schedule.start_time': { $lt: new.end + 30min },
  'event_schedule.end_time': { $gt: new.start - 30min } }
  
```

Figure 4.2 shows an example. An existing event A ends at 08:45. A request B1 starting at 09:00 has a setup window from 08:30, which overlaps A, so it is rejected with `VENUE_BUFFER_COLLISION`. A request B2 starting at 09:15 has a setup window from 08:45, which touches but does not overlap A, and is accepted. Because the extended interval of the new event is compared with the raw times of the existing events, the guaranteed minimum gap between two events is 30 minutes, shared by the teardown of the first and the setup of the second.

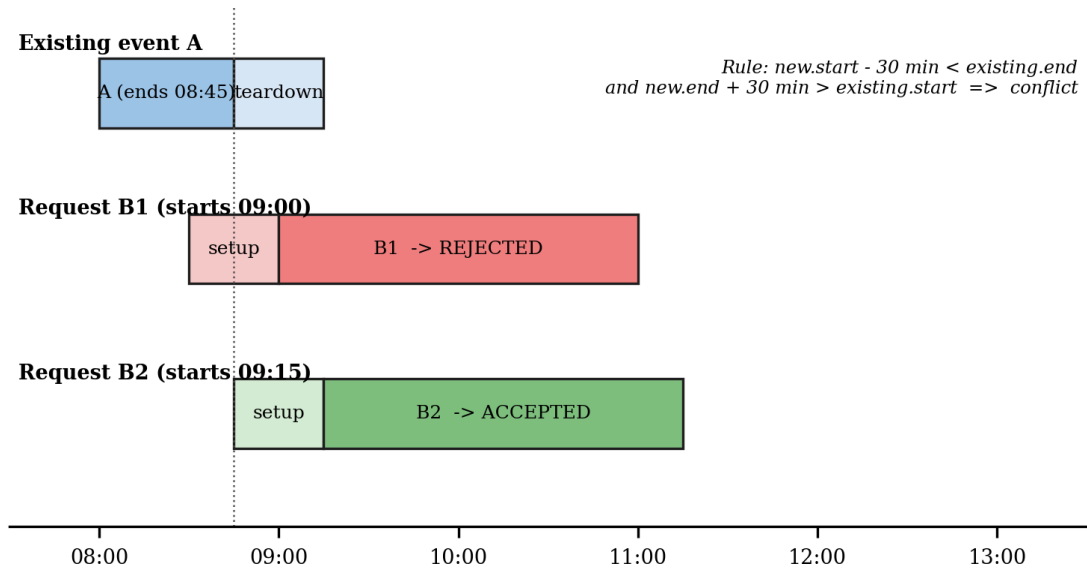


Figure 4.2: Venue buffer collision example

When a conflict is found the response includes the conflicting event, the time when the venue becomes free and a list of alternative venues that have enough capacity and no conflict, so the interface can offer them. To avoid two clubs booking the same slot at the same instant, the check and the approval run in one transaction, or a conditional write on a per-venue booking document is used. The Super Admin can override a conflict with a reason, which is written to the audit log.

4.8 Audit Logging

An `AuditService` provides one method to append an entry with user, role, IP address, action, resource, timestamp and a diff of old and new values. The repository interface has no update or delete method. Entries are written for logins by administrators, approvals, rejections, overrides, denied requests and financial reads. A TTL index keeps entries for 365 days. The Super Admin can query the log with filters.

4.9 Security Hardening

- Rate limits of 100 requests per minute in general and 5 per minute for authentication.
- Strict CORS allowing only the Arya College domains, and security headers on all responses.
- Validation of every request body and path parameter; unknown fields are rejected.
- Uniform error messages for authentication failures; no stack traces in responses.
- Secrets read from environment variables; signing key rotation procedure documented.

- Dependency vulnerability scan in CI and a review against the STRIDE table in Week 13.

4.10 Audit and Ledger Read APIs for the Super Admin

To feed the finance console of Ankit Yadav, I provide read-only endpoints for the campus revenue summary, the per-club totals and the paged ledger and audit lists with filters by club, event, date and type. Each ledger row includes the payment identifier so the interface can link to it. These endpoints are limited to SUPER_ADMIN, and the club-level endpoints to the owning CLUB_ADMIN.

4.11 Transcript PDF and Dean Sign-Off

Using the points data of Anshika Pathak, the service generates the Official Extracurricular and Activity Points Transcript: header of the college and the Dean's office, student metadata, a cumulative table by domain with caps, the itemised log with certificate identifiers, and a QR code for the public verification URL. The Super Admin's sign-off is an API call that records the signer and time, and only signed transcripts show the Dean authorisation mark. The public view exposes only a total and verification status; itemised details need authentication.

4.12 Recommender Scoring API

The recommender implements a transparent hybrid score:

$$S(u,e) = a \times \cos(I_u, G_e) + b \times D(u,e) + c \times H(u,e) \quad (2)$$

where $\cos(I_u, G_e)$ is the cosine similarity of interest and tag vectors, D equals 1 if the event targets the student's department and semester, H is the fraction of the student's past verified attendances with matching tags, and a, b, c are non-negative weights summing to one. For new students with no history the third term is zero and the weights are rescaled. `GET /recommendations` returns the top events with the score and a short reason string for the frontend.

4.13 Error Handling and Response Envelope

All errors are converted by one exception handler into the standard envelope with `status_code`, `error_code`, `message`, `errors` and `timestamp`. Typical codes for my modules are `VALIDATION_FAILED` (400), `INVALID_CREDENTIALS` (401), `TOKEN_EXPIRED` (401), `FORBIDDEN_ROLE` (403), `CLUB_SCOPE_VIOLATION` (403), `EVENT_STATE_INVALID` (409) and `VENUE_BUFFER_COLLISION` (409).

4.14 Testing Approach

Table 4.3: Test plan for my section

Level	Tool	Examples
Unit	JUnit / Jest	Password hashing, token claims and expiry, state-machine transitions, buffer arithmetic
Property-based	Generated intervals	Indexed overlap query equals brute-force check for random intervals
Integration	MockMvc / Supertest with test MongoDB	Register, login, refresh rotation and reuse detection
Authorisation matrix	Automated loop	Every endpoint called with each role and without a token
Security	Checklist and scanner	Rate limit trips at the sixth login; role field ignored on signup
Contract	OpenAPI diff	Frontend mocks match actual responses

4.15 Anticipated Challenges and Planned Solutions

Table 4.4: Challenges and solutions

Challenge	Planned solution
Stolen refresh token	Rotation, reuse detection and family revocation; HttpOnly cookie
Role changes before token expiry	Revalidate role and club scope against the database for financial operations; short deny-list
Two clubs booking the same slot at once	Transaction or conditional write around check and approve
Time-zone mistakes in bookings	Store UTC; convert for display; tests around midnight and daylight-neutral IST
Frontend blocked waiting for APIs	Publish OpenAPI and mock responses in Week 2
Large audit log growth	TTL index, indexes on user and action, paged queries

4.16 Integration with Teammates

Govind Jangid and Ankit Yadav consume my authentication, club, event, venue and audit APIs, so I publish the OpenAPI document and sample tokens early. Ayushi Garg and Anshika Pathak use my role checks and event states: registration requires the APPROVED state, and the points engine runs when I mark an event COMPLETED. Changes to these contracts are announced in the weekly meeting and recorded in the interface table.

CHAPTER 5

14-WEEK WORK PLAN

5.1 My Week-by-Week Plan

Table 5.1 gives my plan for the 14 weeks together with the expected deliverable of each week. The weeks follow the ten phases of the roadmap.

Table 5.1: My 14-week plan

Week	Phase	My tasks	Deliverable / check
1	Setup	Backend scaffold; user and token models; response envelope; specification review	Project builds; error envelope tested
2	1	JWT access/refresh rotation; BCrypt; RBAC guards; rate limits	Login and refresh pass tests; reuse revokes family
3	2	Clubs API with club scoping; read-only treasury endpoint	Club scoping verified by tests
4	3	Event proposal API; DRAFT to PENDING_APPROVAL; audit logging	Proposal saved as pending with unique slug
5	3	Venue buffer engine; conflict API with alternatives	Overlap within buffer rejected with alternatives
6	4	Approval notifications; scheduler for LIVE and COMPLETED	Status changes automatically at start and end
7	5	Audit and ledger read APIs; campus revenue summary	Super Admin sees totals matching ledger
8	6	RBAC matrix tests; security headers and CORS	Authorisation matrix green
9	7	Rate limiting, input validation, audit retention (TTL)	Rate limits and headers verified
10	8	Transcript PDF generator; Dean sign-off API	Signed transcript PDF generated
11	9	Recommender scoring API	Scores and reasons returned
12	10	Integration tests for auth, clubs and events	Integration tests pass in CI
13	Test	STRIDE security review; penetration checklist	Security review closed; issues fixed
14	Pilot	Security sign-off; API documentation	API documentation and security sign-off

5.2 Team Overview and Dependencies

Figure 5.1 shows the whole team schedule. In every week each member has one main deliverable, and the weekly meeting reviews all five. The dependencies between my tasks and the work of the other members are listed in Table 5.2.

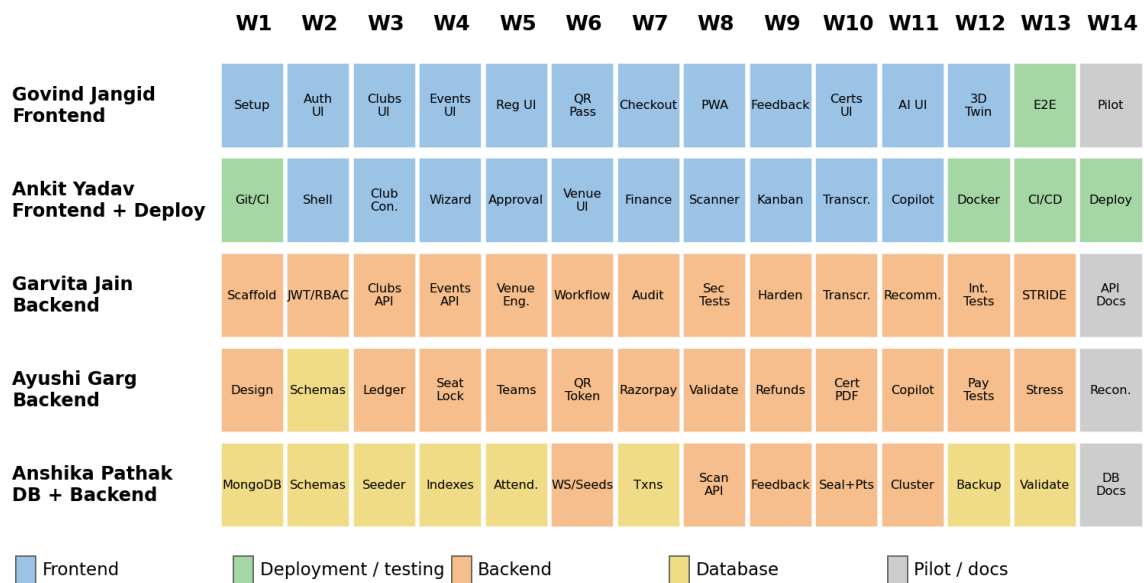


Figure 5.1: 14-week work distribution of the five members

Table 5.2: Dependencies of my weekly tasks on teammates

Week	My task	Depends on / provides to
2	JWT and RBAC	Govind Jangid and Ankit Yadav use role claims for guards
3-4	Clubs and events API	Anshika Pathak: seeded data and indexes
5	Venue engine	Ankit Yadav: conflict UI; Anshika Pathak: indexes
6	Status scheduler	Ayushi Garg: registration needs APPROVED; Anshika Pathak: attendance
7	Audit and ledger APIs	Ayushi Garg: ledger entries; Ankit Yadav: finance console
10	Transcript	Anshika Pathak: points data; Ankit Yadav: download UI
11	Recommender	Govind Jangid: home feed; Anshika Pathak: interests and tags
12-13	Testing and security	All members: endpoints for the authorisation matrix

5.3 Reporting and Review

Every Friday I give the Team Leader a short status note listing completed tasks, tasks in progress, blockers and changes to interfaces. The guide reviews progress at the end of key milestones (Weeks 2, 5, 7, 9, 11, 13 and 14). A slip is first handled by re-ordering tasks within the week and then, if needed, by moving work between sections in agreement with the affected members.

CHAPTER 6

LEARNING OUTCOMES AND SKILLS

6.1 Technical Skills

- Designing stateless authentication with signed tokens, refresh rotation and reuse detection.
- Applying role-based access control with club scoping and defence in depth.
- Designing and validating a state machine for the event lifecycle.
- Designing an interval-overlap algorithm with buffers and supporting it with the right index.
- Writing audit logging that cannot be altered by application code.
- Applying web security practices: rate limiting, validation, CORS and headers.
- Writing automated authorisation tests and contract tests.

6.2 Professional Skills

- Reading and applying a formal specification suite (PRD, SRS, TRD, ERD, API specification).
- Contract-first collaboration between the frontend, backend and database sections.
- Planning work in weekly deliverables and tracking progress with an issue board.
- Writing clear documentation and status reports.

6.3 Teamwork

Because other members depend on my authentication and event contracts, I learned to publish interfaces early and to treat every change as a change for four other people. Reviewing the registration and payment code of Ayushi Garg and the database work of Anshika Pathak also taught me how the same security rules appear at other layers.

6.4 Problem Solving and Debugging

Security defects often hide in the case that nobody tested, such as a valid token for the wrong club. I plan to find them by writing tests as a table of role, endpoint and expected result, by reviewing against the STRIDE table, and by turning every found defect into a permanent automated test.

6.5 Version Control and Communication

Short-lived branches, small pull requests and meaningful commit messages make collaboration smoother and keep the main branch always deployable. Clear and timely communication with the other four members, including early notice of changes to an interface, is as important as writing good code.

CHAPTER 7

CONCLUSION

This individual report presented my part in the CampusSphere project: the identity and security foundation, the clubs, event and venue modules and the audit, transcript and recommender services. The work is divided among five members into two frontend and deployment sections and three backend and database sections, planned over 14 weeks following the ten-phase roadmap.

My modules ensure that only the right person can do the right thing: passwords are hashed, tokens are short lived and revocable, roles are enforced in the services and clubs are scoped. The venue engine removes double booking including the setup and cleanup time, the approval workflow gives the Dean control, and the audit log makes every important action accountable.

This report documents the design and the plan of my work. Performance targets are to be measured in Weeks 12 to 14 and in the pilot. The experience of designing a secure backend foundation and coordinating its contracts with four teammates has given me a strong base for professional backend development.

REFERENCES

- [1]AICTE, “Mandatory extracurricular activity points for B.Tech degree,” All India Council for Technical Education, New Delhi, and Rajasthan Technical University, Kota.
- [2]CampusSphere Project Team, “Product Requirements Document (PRD), v1.0.0-PROD,” Arya College of Engineering & I.T., Jaipur, 2026.
- [3]CampusSphere Project Team, “Software Requirements Specification (SRS), IEEE Std 830 format, v1.0.0-PROD,” 2026.
- [4]CampusSphere Project Team, “Technical Requirements Document (TRD), v1.0.0-PROD,” 2026.
- [5]CampusSphere Project Team, “REST API Specification, v1.0.0-PROD,” 2026.
- [6]CampusSphere Project Team, “Security Architecture and Fraud Prevention Specification, v1.0.0-PROD,” 2026.
- [7]M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
- [8]N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conf., Monterey, CA, USA, 1999, pp. 81–91.
- [9]R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, “Role-based access control models,” IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [10]A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: Wiley, 2014.
- [11]OWASP Foundation, “OWASP Cheat Sheet Series.” [Online]. Available: <https://cheatsheetseries.owasp.org/>
- [12]R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.
- [13]MongoDB Inc., “Indexes and TTL indexes,” MongoDB Manual 7.0. [Online]. Available: <https://www.mongodb.com/docs/manual/>
- [14]Government of India, “The Digital Personal Data Protection Act, 2023,” Act No. 22 of 2023, Aug. 2023.